

Add missing constructors and assignment for indirect and polymorphic

ISO/IEC JTC1 SC22 WG21 Programming Language C++

P3152R0

Working Group: Library Evolution, Library

Date: 2024-09-30

Jonathan Coe <jonathanbcoe@gmail.com>

Antony Peacock <ant.peacock@gmail.com>

Sean Parent <sparent@adobe.com>

Contents

- [Introduction](#)
- [Single-argument constructors](#)
- [Perfect-forwarded assignment](#)
- [Technical specifications](#)
- [Reference implementation](#)
- [Acknowledgements](#)
- [References](#)

Introduction

New vocabulary types `indirect` and `polymorphic` for composite class design are proposed in P3019 [1]. Based on recommendations from LEWG, we follow up this work by adding support for single-argument construction, initializer-list construction and perfect-forwarded assignment to `indirect` and `polymorphic` where appropriate.

The design changes to P3019 are presented in this paper so that they can be reviewed by the Library Evolution Working Group in isolation. Changes to formal wording from this paper have been incorporated into P3019R9 so that wording can be reviewed in its entirety by the Library Wording Group.

Additional constructors

Single-argument constructors

In line with `optional` and `variant`, we add single-argument constructors to both `indirect` and `polymorphic` so they can be constructed from single values without the need to use `in_place` or `in_place_type`. As `indirect` and `polymorphic` are allocator-aware types, we also provide allocator-extended versions of these constructors, in line with those from `basic_optional` [2] and existing constructors from `indirect` and `polymorphic`.

As `indirect` and `polymorphic` will use dynamic memory, the single-argument constructors are marked as `explicit`, the same as other constructors in `indirect` and `polymorphic`.

Initializer-list constructors

We add initializer-list constructors to both `indirect` and `polymorphic` in line with those in `optional` and `variant`. As `indirect` and `polymorphic` are allocator-aware types, we provide allocator-extended versions of these constructors, in line with those from `basic_optional` [2] and existing constructors from `indirect` and `polymorphic`.

As `indirect` and `polymorphic` will use dynamic memory, the initializer-list constructors are marked as `explicit`, the same as other constructors in `indirect` and `polymorphic`.

Perfect-forwarded assignment

Perfect-forwarded assignment for `indirect`

We add a perfect-forwarded assignment operator for `indirect` in line with those from `optional` and `variant`.

```
template <class U=T>
constexpr optional& operator=(U&& u);
```

When assigning to an `indirect`, there is potential for optimisation if there is an existing owned object to be assigned to:

```
indirect<int> i;
foo(i); // could move from `i`.
if (!i.valueless_after_move()) {
    *i = 5;
} else {
    i = indirect(5);
}
```

With value assignment, handling the valueless state and potentially creating a new `indirect` object is done within the value assignment. The code below is equivalent to the code above:

```
indirect<int> i;
foo(i); // could move from `i`.
i = 5;
```

Perfect-forwarded assignment for `polymorphic`

There is no perfect-forwarded assignment for `polymorphic` as type information is erased. There is no optimisation opportunity to be made as a new object will need creating regardless of whether the target of assignment is valueless or not.

Technical specifications

Here we list additions to the technical specifications in P3019 [1] to include the constructors and assignment operators discussed above.

X.Y Class template `indirect` [`indirect`]

X.Y.1 Class template `indirect` synopsis [`indirect.syn`]

```
template <class T, class Allocator = allocator<T>>
class indirect {

    // ... existing constructors
```

```

template <class U=T>
explicit constexpr indirect(U&& u);

template <class U>
explicit constexpr indirect(allocator_arg_t, const Allocator& a, U&& u);

template<class U, class... Us>
explicit constexpr indirect(in_place_t, initializer_list<I> ilist,
                           Us&&... us);

template<class U, class... Us>
explicit constexpr indirect(allocator_arg_t, const Allocator& a,
                           in_place_t, initializer_list<I> ilist,
                           Us&&... us);

// Remaining constructors and assignment ...

template <class U=T>
constexpr indirect& operator=(U&& u);

// Remaining member functions ...
};

```

X.Y.3 Constructors [indirect.ctor]

```

template <class U=T>
explicit constexpr indirect(U&& u);

```

A. *Constraints*: `is_same_v<remove_cv_ref_t<U>, U>` is true. `is_same_v<U, indirect>` is false. `is_same_v<U, in_place_t>` is false. `is_constructible_v<T, U>` is true. `is_copy_constructible_v<T>` is true. `is_default_constructible_v<allocator_type>` is true.

B. *Mandates*: T is a complete type.

C. *Effects*: Constructs an owned object of type T with `std::forward<U>(u)`, using the allocator `alloc`.

```

template <class U=T>
explicit constexpr indirect(allocator_arg_t, const Allocator& a, U&& u);

```

D. *Constraints*: `is_same_v<remove_cv_ref_t<U>, U>` is true. `is_same_v<U, indirect>` is false. `is_same_v<U, in_place_t>` is false. `is_constructible_v<T, U>` is true. `is_copy_constructible_v<T>` is true.

E. *Mandates*: T is a complete type.

F. *Effects*: `alloc` is direct-non-list-initialized with `a`. Constructs an owned object of type T with `std::forward<U>(u)`, using the allocator `alloc`.

```

template<class I, class... Us>
explicit constexpr indirect(in_place_t, initializer_list<I> ilist,
                           Us&&... us);

```

G. *Constraints*: `is_copy_constructible_v<T>` is true. `is_constructible_v<T, initializer_list<I>&, Us...>` is true. `is_default_constructible_v<allocator_type>` is true.

H. *Mandates*: T is a complete type.

I. *Effects*: Constructs an owned object of type T with the arguments `ilist, std::forward<Us>(us)...`, using the allocator `alloc`.

```
template<class I, class... Us>
explicit constexpr indirect(allocator_arg_t, const Allocator& a,
                           in_place_t, initializer_list<I> ilist,
                           Us&&... us);
```

J. *Constraints*: `is_copy_constructible_v<T>` is true. `is_constructible_v<T, initializer_list<I>&, Us...>` is true.

K. *Mandates*: T is a complete type.

L. *Effects*: `alloc` is direct-non-list-initialized with `a`. Constructs an owned object of type T with the arguments `ilist, std::forward<Us>(us)...`, using the allocator `alloc`.

X.Y.5 Assignment [indirect.assign]

```
template <class U=T>
constexpr indirect& operator=(U&& u);
```

A. *Constraints*: `is_same_v<remove_cvref_t<U>, indirect>` is false. `is_constructible_v<T, U>` is true. `is_assignable_v<T&, U>` is true.

B. *Mandates*: T is a complete type.

C. *Effects*: If `*this` is valueless then equivalent to `*this = indirect(allocator_arg, alloc, std::forward<U>(u))`; Otherwise, equivalent to `**this = std::forward<U>(u)`.

D. *Returns*: A reference to `*this`.

X.Y Class template polymorphic [polymorphic]

X.Y.1 Class template polymorphic synopsis [polymorphic.syn]

```
template <class T, class Allocator = allocator<T>>
class polymorphic {

    // ... existing constructors

    template <class U=T>
    explicit constexpr polymorphic(U&& u);

    template <class U=T>
    explicit constexpr polymorphic(allocator_arg_t, const Allocator& a, U&& u);

    template <class U, class I, class... Us>
    explicit constexpr polymorphic(in_place_type_t<U>,
                                   initializer_list<I> ilist, Us&&... us)

    template <class U, class I, class... Us>
    explicit constexpr polymorphic(allocator_arg_t, const Allocator& a,
                                   in_place_type_t<U>,
                                   initializer_list<I> ilist, Us&&... us)

    // Remaining constructors and member functions...
};
```

X.Z.3 Constructors [polymorphic.ctor]

```
template <class U=T>
explicit constexpr polymorphic(U&& u);
```

A. *Constraints*: `is_same_v<remove_cv_ref_t<U>, U>.is_same_v<U, polymorphic>` is false. `derived_from<U, T>` is true. `is_copy_constructible_v<U>` is true. `is_constructible_v<U, U>` is true. `U` is not a specialization of `in_place_type_t`. `is_default_constructible_v<allocator_type>` is true.

B. *Mandates*: `T` is a complete type.

C. *Effects*: Constructs an owned object of type `U` with `std::forward<U>(u)` using the allocator `alloc`.

```
template <class U=T>
explicit constexpr polymorphic(allocator_arg_t, const Allocator& a, U&& u);
```

D. *Constraints*: `is_same_v<remove_cv_ref_t<U>, U>.is_same_v<U, polymorphic>` is false. `derived_from<U, T>` is true. `is_copy_constructible_v<U>` is true. `is_constructible_v<U, U>` is true. `U` is not a specialization of `in_place_type_t`. `is_default_constructible_v<allocator_type>` is true.

E. *Mandates*: `T` is a complete type.

F. *Effects*: `alloc` is direct-non-list-initialized with `a`. Constructs an owned object of type `U` with `std::forward<U>(u)` using the allocator `alloc`.

```
template <class U, class I, class... Us>
explicit constexpr polymorphic(in_place_type_t<U>,
                               initializer_list<I> ilist, Us&&... us)
```

G. *Constraints*: `is_same_v<remove_cv_ref_t<U>, U>.is_same_v<U, polymorphic>` is false. `derived_from<U, T>` is true. `is_copy_constructible_v<U>` is true. `is_constructible_v<U, initializer_list<I>&, Us...>` is true. `is_default_constructible_v<allocator_type>` is true.

H. *Mandates*: `T` is a complete type.

I. *Effects*: Constructs an owned object of type `U` with the arguments `ilist, std::forward<Us>(us)...` using the allocator `alloc`.

```
template <class U, class I, class... Us>
explicit constexpr polymorphic(allocator_arg_t, const Allocator& a,
                               in_place_type_t<U>,
                               initializer_list<I> ilist, Us&&... us)
```

J. *Constraints*: `is_same_v<remove_cv_ref_t<U>, U>.is_same_v<U, polymorphic>` is false. `derived_from<U, T>` is true. `is_copy_constructible_v<U>` is true. `is_constructible_v<U, initializer_list<I>&, Us...>` is true.

K. *Mandates*: `T` is a complete type.

L. *Effects*: `alloc` is direct-non-list-initialized with `a`. Constructs an owned object of type `U` with the arguments `ilist, std::forward<Us>(us)...` using the allocator `alloc`.

Reference implementation

A C++20 (and C++14 compatible) reference implementation of the work discussed in this proposal is available on GitHub at [https://www.github.com/jbcoe/value_types].

Acknowledgements

Many thanks to Neelofer Banglawala for collating information and preparing this draft at extremely short notice.

References

[1] *indirect and polymorphic: Vocabulary Types for Composite Class Design*,
J. B. Coe, A. Peacock, and S. Parent, 2024
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3019r6.html>

[2] *An allocator-aware optional type*,
P. Halpern, N. D. Ranns, V. Voutilainen, 2024
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p2047r7.html>